SN Computer Science (2023) 4:341
https://doi.org/10.1007/s42979-023-01776-3

ORIGINAL RESEARCH

An Efficient Framework for Algorithmic Metadata Extraction over Scholarly Documents Using Deep Neural Networks

P. Raghavendra Nayaka[1] · Rajeev Ranjan[2]

Received: 20 February 2023 / Accepted: 6 March 2023 / Published online: 20 April 2023
© The Author(s), under exclusive licence to Springer Nature Singapore Pte Ltd 2023

Abstract

The conventional text documents have made it possible to efficiently retrieve large amounts of text data with the development of various search engines. However, these traditional search approaches frequently have lower accuracy in retrieval, particularly when documents have certain characteristics that call for more in-depth semantic extraction. A search engine for algorithms called Algorithm Seer has recently been developed. The normal search engine collects the deep textual metadata and pseudo-codes from research papers. However, such a system is unable to accommodate user searches that attempt to identify algorithm-specific information, such as the data sets on which algorithms operate their effectiveness, runtime complication, etc. A number of improvements to the previously suggested algorithm search engine are given in this study. We provide various ways to identify automatically and extract pseudo-codes and phrases which transmit metadata utilizing various machine learning methods. Around the 89,000 text lines are used for conducting the experiments; we provided new properties to extract algorithmic pseudo-codes. These characteristics include feature groups with a focus on content, font style, and structure. Our suggested pseudo-code extraction method outperforms current strategies by 28% and obtains a 94.23% Classification Accuracy. Additionally, we suggest a technique for extracting phrases linked to algorithms utilizing deep neural networks, which achieves an 82% of accuracy compared to recent rule-based provides 23.5% and support vector machine provides 21.5%.

Keywords Scholarly data · Deep neural networks · Machine learning · Algorithm extraction · Metadata extraction

Introduction

As scientific communities' research efforts in university and business rise over time, there has been a rapid increase in the number of scholarly documents produced [1–3]. Millions of articles may now be stored and indexed in digital archives because to improvements in web-based tools and technology.

Researchers now depend on digital libraries and academic search engines such as Springer, Web of Science, Microsoft Academic, and Semantic Scholar to stay current on scientific developments [4, 7]. These scholarly repositories frequently use simple text matching and keyword-based search methods, which fail to predict query/document semantic relevance in some information needs. As a result, a document is often presented as a collection of words without processing [42, 46, 49]. For example, a simple text match between the query and document text might not be sufficient to answer the inquiry "Which model delivers the best accuracy on IMDB data set?" The most of the text-based information gathering algorithms often generate articles that meet the search criteria and could even exclude any mention of techniques used on the IMDB data set. Therefore, intelligent algorithms that match with content scientific publication with pertinent algorithm-specific metadata are necessary for this kind of advanced information. Because they understand a document as a "chunk of full-text" that is afterwards treated as a "bag of words," the majority of

This article is part of the topical collection "Advances in Computational Approaches for Image Processing, Wireless Networks, Cloud Applications and Network Security" guest edited by P. Raviraj, Maode Ma and Roopashree H R.

✉ P. Raghavendra Nayaka
raghavendra.nayak@reva.edu.in

Rajeev Ranjan
rajeev.ranjan@reva.edu.in

[1] School of C& IT, REVA University, Bengaluru, Karnataka 560064, India

[2] School of CSA, REVA University, Bengaluru, Karnataka 560064, India

traditional information retrieval algorithms are unable to provide accurate solutions to such specific algorithmic needs. The conventional document handling fails to fetch the critical data as mentioned by [4, 5].

Many researchers have shared their newly build algorithms through scholarly papers, journal publishing, open-access archival services, conference sessions, and other venues. These scientific articles include pseudo-code (algorithms), as well as pertinent algorithmic circumstances including evaluation findings, data sets utilized by the algorithm and complexity. According to [9], prominent computer science conferences produced about 850 algorithms total. The capacity to recognize these freshly developing algorithms and to extract the information that goes along with them may be necessary for the development of a particular, semantic-enhanced search technique for algorithms drawn from continuously expanding pool of scholarly publications.

Since several times these "standard" methods also are indexed and put online, searching for them combined with their algorithmic particular metadata is remarkably simple. Some of the examples of conventional algorithms are Heap sort method, Dijkstra's algorithm etc.

A customized collection of over 289 common algorithms is also available from the National Institute of Standards and Technology (NIST). Due to the fact that they are both too young and have not yet gained the support of the research and practice communities, freshly proposed algorithms in scientific journals are difficult to control and catalogued. Additionally, there are difficulties with manual duration of these algorithmic creations and their metadata because to the large emergence of these recently produced algorithms reported in scientific articles.

It takes more time and effort to manually search through scientific articles until you find the algorithmic information you're looking for when looking for new algorithms. However, the necessary to read full research paper will be difficult to find desired algorithm. It also deter many novice algorithm researchers from having to look for state-of-the-art algorithmic solutions and force them to separate to more traditional, albeit less innovative, algorithms that are much easier to discover. Additionally, the process may become much more unpleasant if the users lack experience looking for scientific papers and have a tendency to utilize poor search phrases. As a result, automatic algorithm identification and extraction as well as the metadata associated with them could greatly aid an algorithm search mechanism. Now, it is easy to mine algorithms and algorithmic-specific data from comprehensive publications courtesy to search engine prototypes. These methods can identify algorithm expressions and generate a textual summary for every algorithm, yet there are still lots of things that can be done better and that need to be examined. In particular, by including characteristics of pseudo-code content, their extraction method might be much enhanced. Furthermore, metadata relevant to an algorithm could be taken out of the page outlining the algorithm. For instance, algorithm seekers may find it helpful to have pseudo-codes for algorithms and the features connected with them, such as statistics of data set, performance and runtime analysis [8].

Initially it's been t identified that, in addition to the primary text, pseudo-codes mentioned in a variety of font sizes and writing styles in academic articles. Some of the features extraction approaches collect pseudo-code attributes that are being inspected for the purpose of automatically detecting algorithms. Furthermore, we saw that just a small percentage of an algorithm proposal paper's content discussed the algorithm's characteristics. Examples of words that express algorithmic attributes are shown in Table 1 and can be seen in discussions about an algorithm's assessment findings, experimental data set, and run-time complexity. To give extracted algorithms more algorithmic metadata, our research aims to recognize and extract these kinds of words. Given that algorithmic metadata typically consists of connections to web resources from scholars, academics, and semantic-scholars in the context of text written in a variety of writing styles.

It is impossible to capture the complete semantics associated with algorithmic information utilizing keyword-based techniques for sentences identification that are linked to algorithms across different authors and academic disciplines.

The identification of algorithms expressed utilizing pseudo-codes and to extract metadata about algorithms from a scientific publication, we also provide a collection of special machine learning techniques in our survey. Also provided is a prototype for a search system that concentrates on identifying algorithms suggested in journal articles. Such a system will make use of the suggested machine learning techniques to fetch algorithms and algorithmic information. That is utilized to improve semantic annotation of the associated algorithms, which boost the semantic search capabilities.

Various contributions of the research work are:

- To identify algorithm in scholarly work, we provide a cutting-edge machine learning-enabled technique.

Table 1 Performance analysis of each deployed classifier

Classifier	Precision	Recall	*F*-measure
KNN	0.62	0.63	0.65
Logistic regression	0.51	0.86	0.34
Decision tree	0.72	0.85	0.78
SVM	0.75	0.82	0.86
Proposed approach	0.85	0.92	0.94

- We formulate the issue as a binary categorization task, in which each text line is grouped according to the fact that it is a part of a pseudo-code.
- The 58 new characteristics of document splits into three groups, namely, content, fonts and the structure, in addition to these the machine learning techniques such as LR, SVM, DT, and KNN are applied to classify the content based on various classifiers features.
- The suggested method is tested on 256 research publications that were chosen at random and retrieved from the CiteSeerX repository. We showed that RF outperformed the current state-of-the-art approaches by 28% and achieved a 93% F1-score.
- We propose deep neural network-based categorization method for finding algorithmic metadata sentences that address algorithmic performance assessment, data sets used, and runtime complexities.
- Here, sentence from an algorithm proposal paper is sorted as categories relevant to algorithms, classification is treated as task.

Here, the proposed approach consider every format of algorithmic metadata as a distinct category and produces word-level streams that accurately characterize these phrases with temporally improved algorithmic metadata. To evaluate this proposed approach in relation to two baselines—the rule-based classifier and an SVM classifier—we also perform a number of thorough investigations. About 37,000 sentences from 256 academic papers that were acquired from the CiteSeerX repository make up the input data set. The proposed deep learning algorithm enhances the rule-based and SVM approaches in terms of accuracy by 56.69% and 25.80%, respectively.

We demonstrate the Enhanced-IR System, a discovery system prototype that is algorithm-focused and uses our algorithm metadata extractor. 21,940 full-text articles that were downloaded from the Association of Computational Linguistics (ACL) repository were used in a case study that we deployed. In every text document, we first produce a specific algorithm-based summary called a document synopsis. Additionally, we added algorithmic metadata that was extracted by our suggested deep learning model to these synopses to enhance them. Last but not least, we index and search the retrieved algorithmic metadata using Apache Lucene. Tests have been conducted to contrast proposed enhanced-IR system with traditional TF-IDF-based system.

The rest of the paper is structured as follows. Sect. "Literature review" of this article talks about related work. In Sect. "Proposed methodology", we use machine learning and deep learning models to discuss our data set and suggested technique. Sect. "Results and discussions" provides a detailed explanation of the testing and our obtained results, followed by a thorough analysis of our suggested deep learning search engine. Sect. "Conclusion" concludes with a review of this research's potential uses and future directions.

Literature Review

This section describes various research works carried out on various machine learning techniques used to extract the metadata information from the scholar's data.

Several studies have been conducted to examine how to effectively retrieve various document elements from an article's whole body text, including tables, figures and pseudo-codes [35, 39]. Several rule-based and ML-based algorithms are suggested and implemented and are used for element retrieval task.

The Algorithm Seer [49], a search engine for algorithms, stands out among these, since it automatically pulls algorithms from research publications using a hybrid machine learning technique. The system uses a machine learning approach to learn from the 47 features that are directly produced from each sparse section of a text. The system uses a synopsis-creating method to gather crucial textual information about these approaches after first classifying each region as a pseudo-code or not using the classification model. Around 20,000 research articles provided to help with algorithm-related search difficulties and enhance the search keywords through case studies.

It was also suggested to use the specialized chemical search engine Chemxseer for the extraction of figures, tables and related metadata [29]. The technology makes it easier for scientists and researchers to look up chemical formulas that have been published in indexed chemistry journals. Along with this work, they later created TableSeer, a search engine for tables. AckSeer [19, 20] is a digital library-based acknowledgment search engine. It extracts response from full-text documents using regular expressions. Then, using named entity recognition (NER) technologies, entities are recovered from the acknowledgements. Such an application would involve categorizing academic books into different mathematical categories according to the mathematical expressions they contain, like those from linear algebra, probability theory, and graph theory [48].

Information derived from the articles' photos, charts, and graphs may be necessary to answer some IR queries. Utilizing a computer vision methodology by dividing the axis, their labels, legends, and data points some attempts made to retrieve features from these plots [1, 10]. These initiatives shown that fetching numbers from PDF files is a difficult undertaking, mostly because there are not enough annotated data sets to build reliable models. By categorizing a substantial data set and establishing the neural network for figure extraction, this is made achievable [44, 45].

A cutting-edge technique for figure extraction called Figure Seer was suggested. It mines information from figures in academic publications based on ResNet [45]. Prior to classifying each candidate as a figure or not, the algorithm first locates probable figure candidates. This method also addresses distortion and clutters, producing a better, more precise and potent search. An adaptive method for detecting plagiarism from independent text elements included in academic texts, such as pictures, diagrams, and flowcharts has been developed [26]. The suggested approach incorporates cutting-edge image analysis methods, including position-aware OCR-based approaches and picture similarity assessment. The content-based analysis strategy for identifying academic plagiarism can be supported by this state-of-the-art methodology has been demonstrated by the obtained result.

Semantic links between entities have rarely been explored in more recent days. Through doing everything with human data tagging for training [1, 29], developed an unsupervised technique to building the entity-relationship graph. In an iterative process, this data is produced as a semantic learner gains comprehension from earlier retrieved tuples. To build a relationship for better analysis, the nodes serve as entities. These graphs were used to rank the documents according to their relationships in the graph or for algorithmic discovery.

For many years, text classification has been a hot topic in research. Various machine learning models are used for document categorization problems [18, 30, 36]. The efficiency and efficacy of these models for multi-label classification rapidly decline, which is a significant negative. Deep neural networks and recent developments in representation learning have developed novel techniques for learning textual representations [23, 33]. In contrast to conventional bag-of-words-based models, deep learning techniques such as CNN [21], RNN [23], RCNN [47], and LSTM [13] appear to be particularly effective in text summarization.

On massive data sets, proposed character-level convolutional neural networks for multi-class text classification. The methods were contrasted with popular models as LSTM, BOW, N-grams, TF-IDF, deep learning word-based CNN, and BOW. They discovered for short sample sets, the baseline models were superior to the suggested deep learning models, however, for big sample sets; character level deep learning approaches will be more effective. Deep architectures for text categorization were proposed [12] and were influenced by deep networks for computer vision. The networks use tiny convolutions and pooling layers, and they function directly at the character level. They also disagreed on whether or not deep Convolutional layers improved text categorization performance in deep learning models.

Proposed Methodology

For automatically identifying pseudo-codes (PCs) and collecting algorithmic metadata terms from studies, we provide a variety of machine learning-based techniques. We formulate these issues as text categorization issue in which every text region or sentence is categorized according to whether it contains pseudocode or algorithmic information. The section contains the specifics of our suggested strategies, which are shown in Fig. 1 at a high level. The PDF files were first transformed into text files. Second, from the scientific literature, we derived algorithmic PCs. Then, from those articles that at least contained one or more algorithmic PCs, we extracted algorithmic metadata. To enhance algorithm-focused search results, we provide an example for advanced search engine using the suggested algorithm-specific metadata extraction.

Pseudocode Extraction in the Algorithm

Here, we go into the specifics of the machine learning-based method to extract PC from research publications.

First, Java-based Apache PDFBoX API is used to transform PDF articles into text documents. It facilitates the removal of graphic graphics from a PDF document and supports the extraction of Unicode text. Drawing from Hassan's

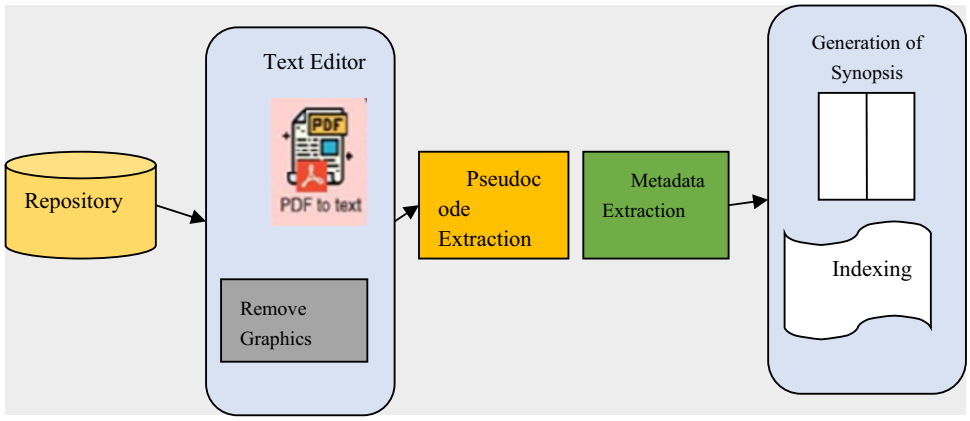

Fig. 1 Proposed flow chart

work, we modified the PDFBoX library to extract object data from a PDF document [14]. Only the textual content and its font information are maintained after this operation to facilitate text mining algorithms in the future. Information about the text and figure alignment is removed. Then, to identify Computer lines in a certain scientific publication, 60 properties are taken from each text line and fed into classification models.

Scientific publications typically present their suggested algorithms as PCs. A set of detailed instructions for resolving a computer problem is what is meant by a PC. Note that since mathematical expressions do not repeat step-by-step computational instructions, they are not regarded as personal computers. These systems are viewed as separate page elements having identifying identifiers that are not included in the running text, such as captions, function names, and algorithm names. The names of their identifiers serve as references for Desktops in the text document, because they can exist anywhere in a scientific article. Here, using data extracted from PCs based on font style, text structure, and context, we define a set of distinctive traits that distinguish flowing text with body of text in scientific publications. The designed model's details are listed below.

Spare Region Identification in the Algorithm

We originally took images of the bare regions to locate PCs from documents. Because these PCs will not be visible, a regular expression-based technique that only looks for captions on PCs often has low recall [49].

In addition, we looked at the PCs in our data set and discovered that 25.8% of them were missing their corresponding PC captions. We began by noting the fact that a PC is frequently written sparsely within a document.

Figure 2 displays examples of sparse areas on a PDF file and related raw-text pages.

A group of M consecutively sparse lines makes up a sparse boX. A line that meets the following criteria is considered sparse: on the line, there should not be a header or footer.

Pseudocode Features Identification

We gathered a diverse set of features, including features based on structure (ST), content (CN), and font style (FS). The attributes that were retrieved along with their descriptions are described. A total of 58 features—16 CN, 34 ST, and 8 FS—were retrieved. Captions and the existence of terms linked to algorithms are both recorded by the CN feature set. Like, FS-based features, the font families, font widths, and indentation types utilized in PC assembly are all examined. The ST characteristics examine the character types and symbols used in the text Sparsity. To prevent model over fitting, we also used a variety of feature normalization approaches.

Metadata Extraction in the Algorithm

The suggested algorithmic metadata extraction approach makes use of deep learning classification techniques to locate text lines in a given document. That describe the suggested algorithmic PC's metadata. Each sentence in an article is categories as algorithmic metadata groups or a class unrelated to algorithms, we specifically used two bi-directional LSTM architectures and a character Convolutional neural network. Despite the reality that word embedding-based methods [27] are suitable for domain-specific tasks, acquiring prior knowledge is still an expensive, because

Fig. 2 Pseudocode extraction from a sparse region in the algorithm

```
Algorithm: Network graph of social media users

    Input: Social media user's profile data set.

    Output: Network Graph of Social Media users.

    Initialization: Initialize Matrix M (similarity matrix) to zero
Begin
    Step 1: For every i^t profile of the data set.
    Step 2: For every j^t profile of the data set.
    Step 3: Compute the Similarity value S_ij between i^t and j^t profile on all attribute A_k
            S_ij (A_k) = (( i^th € j^th attribute)  ^ (word match%)) > Threshold value
    Step 4: M_ij =∑ S_ij (A_k)
    Step 5: End of Step 2
    Step 6: End of Step 1
    Step 7: Construct Network Graph using M_ij matrix
End
```

structural parsers need to handle many different factors, such as morphological shifts and undefined segmentation. The text comprehension is essentially customized to a single language; if a language changes, many rules must be designed from beginning.

Deep learning algorithms can comprehend text scripts without intentionally incorporating knowledge from outside sources regarding words, sentences and other language-related syntactic and semantic ideas [23]. It's crucial to note that all words in the English dictionary were generated using 26 different alphabets. Thus, any word vector may be generated using character inclusion, even from strange words. The only phrases that can be handled by word embeddings are those that are displayed. Characters embeddings are effective for both newly invented words and misspelt words. Due of word embedding vectors' concern about just the lack of training data for uncommon keywords, it handles unusual words significantly improved than word2vec.

Character-based models are also highly effective with time complexity, because they have a limited number of vectors [22]. The intricate architecture of our suggested strategy is shown in Fig. 3. To extract text, the PDF documents must first be parsed. After that, we cleaned up any remaining text with noisy and classified every chunk of text into its appropriate portion using document segmentation and pre-processing. In addition, the deep learning model is provided with the cleaned phrases for the classification of algorithm-specific metadata. For effective document searching, we merged the retrieved algorithm metadata sentences with phrases that specifically mention an algorithm (such "Our proposed strategy is outlined in Algorithm 2").

The suggested model reads each sentence in a document, categorizes it, and then outputs the results. The sentence encoding element is initially provided the text character streams to construct sentence level embedding. To keep track of the placements of words and characters within an embedded text, keep in mind that the LSTM is placed during the encoding phase. Another bi-directional LSTM receives these newly acquired sentence encodings to capture the context and meaning of the sentences. Finally, we included an output layer made up of neurons. The following sections go into detail about each component of the model architecture.

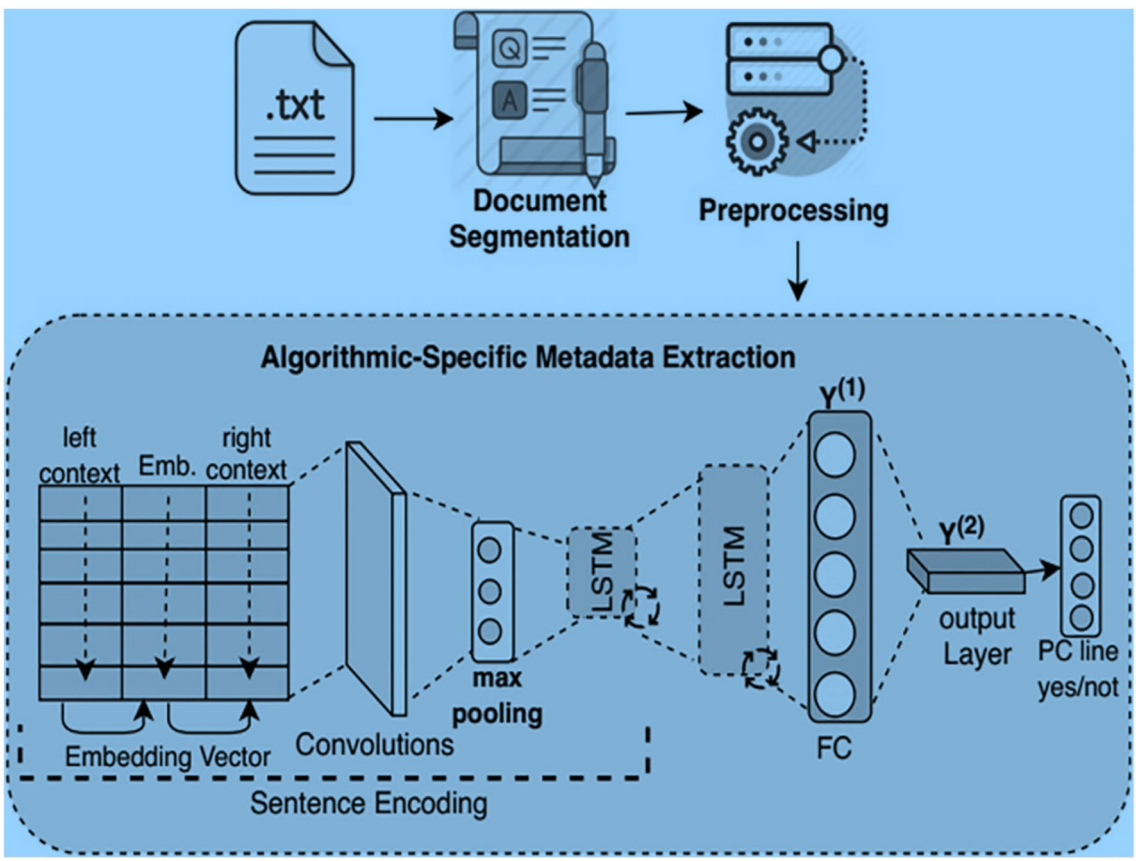

Fig. 3 Proposed system architecture

Classification and Integration of Sentences

The suggested approach accepts the character one-hot encoding techniques as an input vector. From a character set of 70 that contains 26 English alphabets, 10 numbers, new line and 33 more special characters, the vector is constructed by selecting m language alphabets. Then, a one-hot vector is used to represent each character (size of m). Keep in mind that symbols like space that are not part of the set are encoded as zero vectors. Each character receives a 16-dimensional vector in the output.

At the embedding stage, the CNN (convolution and max pool layer) initially reads the characters (one-hot encoding) to produce a series of 0/1 vectors that represent the phrase. Given that sentences might vary in length, we use the LSTM layer to condense the series of vectors into a single fixed-size vector to prevent CNN from representing sentences inconsistently. In greater detail, two bidirectional LSTM layers receive a succession of vectors from the CNN, and the encoded sentence is made by concatenating the most recent output states. The final illustration is a vector that preserves word (character) ordering information while attempting to show word (character) dependencies in both the forward and backward directions. We have used fully connected layers to convert the LSTM output into a probability distribution over the classes in classifying a text. Our categorization model produces the class with the highest likelihood.

Proposed System Architecture

We have the following feature in our model. The character-level CNN consists of a single Convolutional layer with 256 filters and a filter width of 3. This step is represented by an approx layer with a kernel width of 21. We employ two layers, each with 256 and 128 hidden units, for the bidirectional LSTM layers. Last but not least, there are two layers in the fully connected layers. The ReLU activation function used by the first fully connected layer, which comprises 100 units, is shown in the following equation:

$$Y_t = \mathrm{ReLu}\left(D^2 y^1 + b^2\right), \tag{1}$$

where xi is the encoded version of the input data that the LSTM learned. Equations (2) and (3)'s output classes' probability distributions are computed by the last layer's softmax activation function:

$$Y^2 = w^2 y^1 + b^2. \tag{2}$$

$$P_{(i)} = \frac{\mathrm{expr}(y^2)}{Y^2}. \tag{3}$$

Results and Discussion

Here, we present the description of our testing and the purpose of effective for algorithmic and PC extraction of metadata. Along with our suggested algorithmic metadata extracting methodology, also included a crucial baseline methodology known as Rule-based technique and an ML method, so that you can contrast their outputs with ours. Finally, to demonstrate the effectiveness and usability of suggested methods, we show a model of an upgraded scholarly search engine.

Data Sets Considered

In this study, we used two data sets, one for assessing our improved search engine and the other for pseudo-code recognition and algorithmic metadata extraction.

The first data set consists of 256 PDF publications that were initially retrieved from the CiteSeerX repository and contain 282 distinct algorithms and 275 pseudo-codes. These PDF files were chosen at random from the collection of documents that had the word "algorithm" used at least once. We have considered around 93,565 text lines from the data set and manually annotated them to classify them as PC or non-PC lines to identify pseudo-code (PC). It has been observed that just 8% of all lines are classified as PC lines (Class 1), and remaining 92% being non-PC lines (Class 0). Only 92 out of 256 publications have one or more PCs and the textual metadata that goes with them. Therefore, to validate the algorithm's metadata extraction, these 92 publications were used.

To eliminate textual traces and unnecessary lines, we preprocessed about 85 study publications. We are finally left with 32,000 sentences, of which 6.8% have positive annotations (algorithmic metadata sentence) and the remaining 93.2% have negative annotations (non-algorithmic metadata sentence). The positive occurrences, 6.8% (2517), contain three different forms of algorithmic-specific metadata. (1) 128 sentences discussing data sets in a particular document, (2) 2245 sentences discussing algorithmic efficiency, and (3) 76 sentences describing algorithmic time complexity. This data set was intended to assess how well our suggested method for algorithmic information extraction performed.

More documents are required to create a search system. The second data set includes 1490 distinct algorithms spread among 21,848 ACL articles. For the purpose of testing the performances of the proposed better searching engine, we used to use this data set as a study case.

Performance Evaluation

Decision Tree, Linear Logistic Regression, Naive Bayes, SVM, and Linear Logistic Regression are among machine classifiers we used (DT). Because the preliminary reports

for the other classifier were lower in comparison with other tree-based classifiers because of the rule-extracted features. All classifier were exposed to fivefold cross-validation throughout model training. We established standards for RF to balance class weights. To assess the efficacy of each feature type, we also generated mean PR-curves and feature-based PR-curves. These curves go into greater detail on the model's behavior and performance across all features and feature groups.

The PR-curves and AUCPR values for the top five classifiers using fivefold cross-validation are shown in Fig. 4. To learn more about how each feature type affects overall performance. With 0.98 AUCPR employing all features, RF outperformed all other classifiers, we discovered. In addition, KNN and DT both attained 0.78 AUCPR. The worst-performing classifiers, LG and NB, remained to have AUCPRs of 0.65 and 0.09, respectively. It is crucial to understand that NB is a conditional probability model that performed the best when the attribute values are discrete.

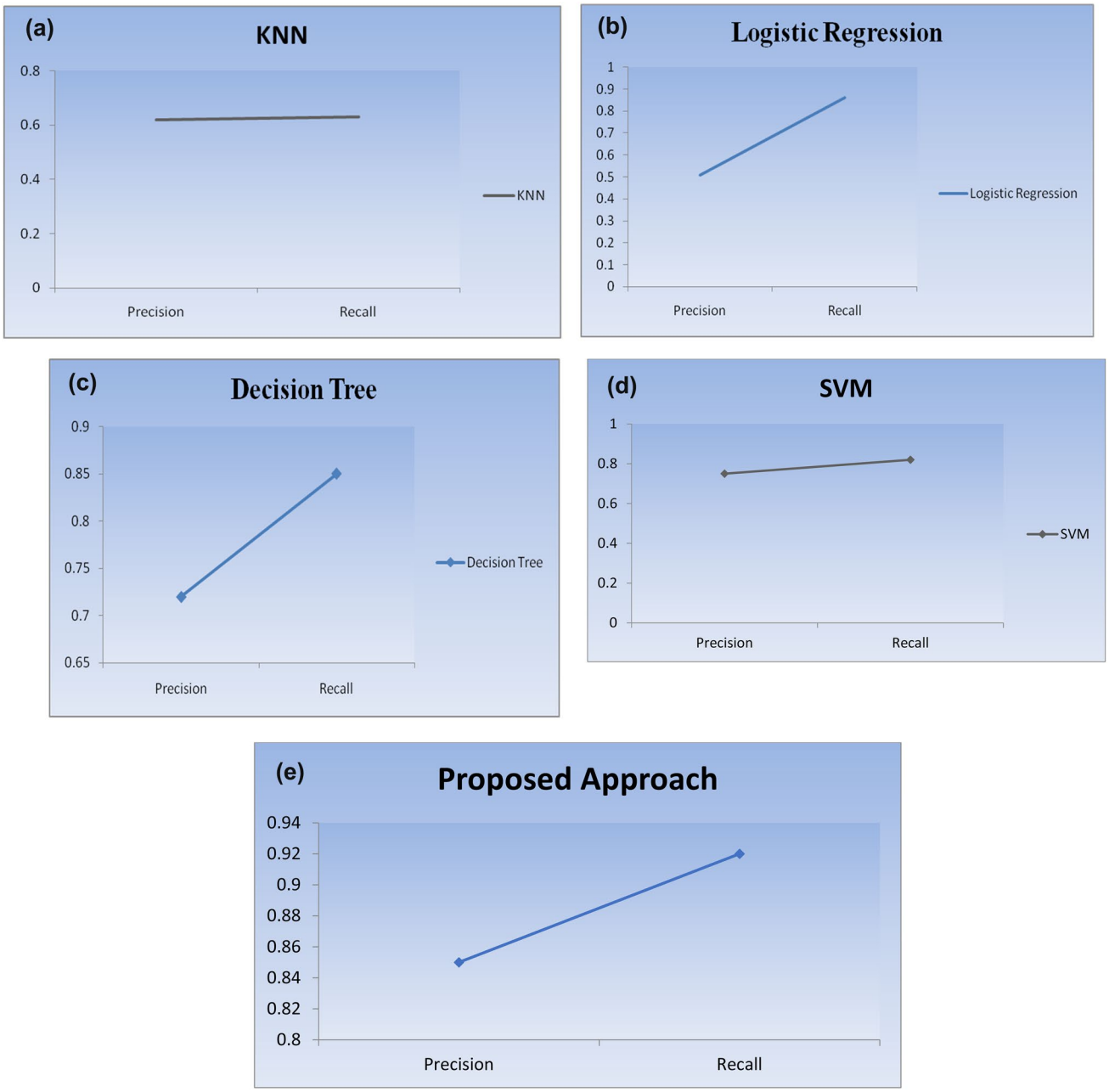

Fig. 4 All classifiers with mean and feature-based PR curves. **a** PR-curve for Knn, **b** PR-curve for LR, **c** PR-curve for DT, **d** PR-curve for SVM, **e** PR-curve for proposed approach

Furthermore, zero conditional distribution is a concern (division by zero). As a result, when applied to our data set's continuous variables, it underperformed.

In addition, using fivefold cross-validation, Table 1 and Fig. 5 describes the precision, recall, and F1-score for each deployed classifier. The study demonstrates that RF is the model that performs the best overall due to its ability to manage many decision trees simultaneously. The conclusion will be average of the decision trees. Because the single decision tree has a more variance score and is susceptible to over fitting, averaging lowers variability and minimizes model over fitting. Furthermore, RF chooses which portions of the data to learn from; as a result, it performs well even on big data sets. In addition, it can deal with class imbalance by making an effort to reduce overall error rates to improve model performance:

$$\text{Precision} = \frac{\text{TP}}{\text{TP} + \text{FP}} \tag{4}$$

$$\text{Recall} = \frac{\text{TP}}{\text{TP} + \text{FN}} \tag{5}$$

$$F - \text{measure} = \frac{2 * \text{Recall} * \text{Precision}}{\text{Recall} + \text{Precision}} \tag{6}$$

In addition, we include the feature-based PR-curves for each classifier's KNN, LR, DT, SVM and proposed approach feature groups, as shown in Fig. 5. With an AUCPR of 0.98, it has been noticed that CN-based features set performs best compared others. This is so that these characteristics can record PC contents, such as algorithm/PC keywords and the presence of captions. In addition, both the FS and ST features have independently produced encouraging outcomes.

In addition to that, fivefold cross-validation is used to evaluate our top-performing classifier RF trained with our suggested parameters to cutting-edge pseudo-code extraction methods already in use. Our PC filtration prototype (PC-ML60) performs better the most innovative (PC-BL) rule-based technique, machine learning-based (PC_LL) and both combined PC_ML approach in terms of both the accuracy, recall and the F-measure values are (90.02%, 90.72% and 92.32%, respectively). The proposed approach is excelled the baseline model adding 9.3% to the precision, 32.23 to the recall and 23.23 to the –measure value, respectively.

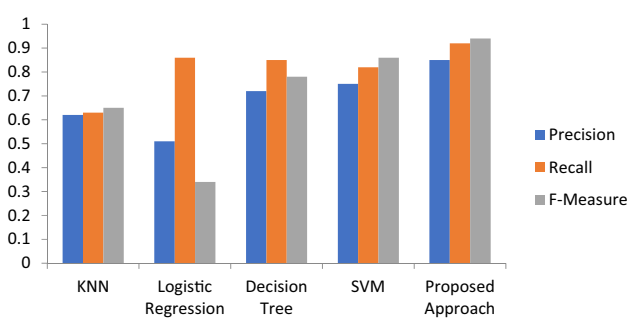

Fig. 5 Performance analysis of each deployed classifier

Rule-Based Method

We created a keyword-based strategy for a baseline benchmark. Using this technique, we painstakingly retrieved from a subset of our data set the keywords associated with algorithmic metadata, such as data set, corpus, execution time, complexity, precision, f-score, etc. In addition, we used a straightforward keyword-matching algorithm to designate every line that matched as an algorithmic-specific metadata text line. For example, a line would be labeled as belonging to the positive class if it contained any of the algorithmically defined keywords. The remaining lines are handled as instances of the negative class. Due to its inability to recognize the context of the phrases, this classical keyword matching technique had very low recall and low precision. The proposed approach is compared with the existing SVM technique with kernel value. Table 1 describes the performance evaluation of both existing and the proposed deep neural network with classification accuracy, precision, recall and F-measure values. With 82% accuracy, our suggested method excelled rule-based and SVM techniques as it can comprehend semantic and contextual descriptions during classification. It's also crucial to keep in mind that, in comparison with the other baselines, the bi-directional LSTM treats an input sentence as a series of words, taking the terms' ordering under consideration. A scientific document's term orderings may serve as useful signals for identifying sentences that include algorithm-specific information, according to the bi-directional LSTM's good classification performance.

Conclusion

In computer science and related disciplines of study, algorithms are used everywhere. It may be helpful for researchers and software engineers working in scientific computing to be able to extract and index algorithms and their metadata from scientific publications to uncover cutting-edge algorithmic solutions to some issues that need to be addressed. Furthermore, both the private and public sectors, particularly academics, can benefit from acquiring the skills and knowledge required to conduct complicated inquiries and provide exceptionally exact outcomes. In this study, a novel machine learning-based approach for locating algorithmic pseudo-codes in scientific articles was suggested. This method is based on 58 distinct features. In addition,

we discovered algorithmic metadata words that, when utilized in conjunction with deep neural networks; enhance an algorithm-focused IR system's search capabilities. Future character embedding-based PC extraction methods will be created using a new set of ETL pipelines. Because they can only read a specific PC line, the current embedding-based techniques are not able to carry out this task satisfactorily. As a result, they cannot understand the contents of a PC box that is separated from the main text of a paper published and is released in a diverse array of font sizes, styles, and spans across multiple paths. In addition, to comprehend the text semantics utilizing alphanumeric letters, we intend to investigate NLP-based techniques, such as tree parsing, etc. (Batista-Navarro et al. [7]). As was already indicated, the public and private sectors would be particularly interested in this, not least, because it would help with information retrieval, comparison, match results, etc.

Funding No funding received for this research.

Data availability The dataset generated and analyzed during the current study are available from the corresponding author on reasonable request.

Declarations

Conflict of Interest No conflict of interest.

References

1. Al Zaidy RA, Giles CL. A machine learning approach for semantic structuring of scientific charts in scholarly documents, Twenty-Ninth IAAI Conference. 2017.
2. Altınel B, Ganiz MC. Semantic text classification: a survey of past and recent advances. Inf Process Manage. 2018;54(6):1129–53.
3. Ramanaidu S, Thompson, Nawaz R. Enhancing search: Events and their discourse context. In: International Conference on Intelligent Text Processing and Computational Linguistics. 2013; Berlin: Springer, pp 318–334.
4. Arshad N, Bakar A, Soroya S, Safder I, Haider S, Hassan S, Aljohani N, Alelyani S, Nawaz R. Extracting scientific trends by mining topics from Call for Paper, Library Hitch. 2019; https://doi.org/10.1108/LHT-02-2019-0048.
5. Azad HK, Deepak A. Query expansion techniques for information retrieval: a survey. Inf Process Manage. 2019;56(5):1698–735.
6. Al Zadran, Giles CL. Extracting semantic relations for scholarly knowledge base construction. In: 2018 IEEE 12th international conference on semantic computing (ICSC); 2018. pp. 56–63.
7. Batista-Navarro RT, Kontonatsios G, Mihaly C, Thompson P, Nawaz R, Mihaly L, Ramanaidu S. Facilitating the analysis of discourse phenomena in an interoperable NLP platform. In: International Conference on Intelligent Text Processing and Computational Linguistics. Berlin: Springer 2013; pp. 559–571.
8. Bakar A, Sederma, Hasan's U. Mining algorithmic complexity in full-text scholarly documents. In: ICADL Poster Proceedings the University of Waikato. 2018.
9. Bhatia's, Mitra. Summarizing figures, tables, and algorithms in scientific publications to augment search results'. Trans Inform Syst (TOIS). 2013;30(1),3.
10. Giles. Curves parathion for line graphs in scholarly documents. In: Proceedings of the 16th ACM/IEEE-CS on Joint Conference on Digital Libraries. 2016+; pp. 277–278.
11. Clark, Divola's. Mining figures from research papers, in: Digital Libraries (JCDL). In: 2016 IEEE/ACM Joint Conference. 2016; pp. 143–152. IEEE.
12. Conneau A, Schwenk H, Barrault L, Lecun Y. Very deep convolutional networks for text classification. ECACL. 2016;1:1107–16.
13. Greff K, Srivastava RK, Koutnik J, Steunebrink BR, Chmidhuber J. LSTM: A search space odyssey. IEEE Trans Neural Netw Learn Syst. 2016;28(10):2222–32.
14. Hassan T. Object-level document analysis of PDF files. Proceedings of the 9th ACM symposium on Document engineering 1. 2009; pp. 47–55). ACM.
15. Huang M, Qian Q, Zhu X. Encoding syntactic knowledge in neural networks for sentiment classification. ACM Trans Inform Syst (TOIS). 2017;35(3):26.
16. Imran M, Akhtar A, Said A, Safder I, Hassan SU, Aljohani NR. Exploiting social networks of Twitter in alt metrics big data. In: 23rd international conference on science and technology indicators (STI2018) Centre for Science and Technology Studies (CWTS) September12–142018 Sep11. 2018.
17. Jahangir M, Afzal H, Ahmed M, Khurshid K, Nawaz R. An expert system for diabetes prediction using auto tuned multi-layer perceptron. In: 2017IntelligentSystemsConference (IntelliSys). 2017; pp. 722–728. IEEE.
18. Joachims T. Text Giles. C.L.: Learning with many relevant features. European conference on machine learning. 1998; Springer, pp. 137–142.
19. Jorge A, Springer. June Iterative part-of-speech tagging. In International Conference on Learning Language in Logic. 1999; pp. 170–183.
20. Giles CL, Treatise M, Giles CL. Ackseer: a repository and search engine for automatically extracted acknowledgments from digital libraries. 2012
21. Kim Y. Convolutional neural networks for sentence classification. EMNL P. 2014; 1746–1175.
22. Kim Y, Jernite Y, Sontag D, Rush AM. Character-aware neural language models. In: Thirtieth AA AI Conference on Artificial Intelligence. 2016.
23. Lai, S., Xu, L., Liu, K., &Zhao, J. (2015). Recurrent convolutionalneural networks for text classification. Twenty-ninth AAAI conference on artificial intelligence.
24. Li X, de Rinke M. Characterizing and predicting downloads in academic search. Inf Process Manage. 2019;56(3):394–407.
25. Mai Gaulke L, Scherm A. Using Deep Learning for Title-Based Semantic Subject Indexing to Reach Competitive Performance to Full-Text. In: Proceedings of the18thACM/IEEE on Joint Conference on Digital Libraries. 2018; pp. 169–178. ACM.
26. Mischke, Ngoma, Searcher, Beijinger, Kimd, Kimd. An adaptive image-based plagiarism detection approach. In: Proceedings of the18thACM/IEEE on Joint Conference on Digital Libraries. 2018; pp. 131–140. ACM.
27. Manolov, Manolov, Chunk, Caradog's, Dean. Distributed representations of words and phrases and their compositionality. In: Advances in neural information processing systems. 2013; pp. 3111–3119.
28. Militia RL, Labor ES, Pessoa AA. A work-efficient parallel algorithm for constructing Huffman codes. Proceedings DCC '99 Data Compression Conference. 1999; pp. 277–286. IEEE Cat. No.PR00096.
29. Mitra P, Giles CL, Snub, Lucy. Chem Seer: a digital library and data repository for chemical kinetics. In: Proceedings of the ACM

first workshop on Cyber Infrastructure: information management in e Science. 2007; pp. 7–10. ACM.
30. Mahmood Z, Safder I, Nawab RMA, Bukhari F, Nawaz R, Alfaki AS, Hassan SU. Deep sentiments in Roman Urdu text using Recurrent Convolutional Neural Network model. Inf Process Manage. 2020;57(4): 102233.
31. Nawaz R, Thompson P, Ramanaidu S. Identification of Manner in Bio-Events. LREC. 2012; pp. 3505–3510.
32. Petrakis E, Georgiadis C. Evaluation of spatial similarity methods for image retrieval. In: Conference on Signal Processing Communications and Computer Science. 2000; pp. 13–18.
33. Rastan R, Paik HY, Shepherd J. TEXUS: a unified framework for extracting and understanding tables in PDF documents. Inf Process Manage. 2019;56(3):895–918.
34. Rubin TN, Chambers A, Smyth P, Stivers M. Statistical topic models for multi-label document classification. Mach Learn. 2012;88(1–2):157–208.
35. Rush AM, Harvard SEAS, Chopra S, Weston J. A neural attention model for sentence summarization. ACL Web. In: Proceedings of the 2015 conference on empirical methods in natural language processing; 2017.
36. Safder I, Hassan SU. Bibliometric-enhanced information retrieval: a novel deep feature engineering approach for algorithm searching from full-text publications. Scientometrics. 2019;119(1):257–77.
37. Safder I, Hassan SU. DS4 A: Deep search system for algorithms from full-text scholarly bigdata. In: 2018 IEEE International Conference on Data Mining Workshops (ICDMW). 2018; pp.1308–1315.
38. Safder I, Hassan SU, Aljohani NR. AI cognition in searching for relevant knowledge from scholarly big data, using a multi-layer perceptron and recurrent convolutional neural network model. In: Companion Proceedings of the Web Conference 2018. pp. 251–258. International World Wide Web Conferences Steering Committee.
39. Safder I, Sarfraz J, Hassan SU, Ali M, Taarab S. Detecting target text related to algorithmic efficiency in scholarly big data using recurrent convolutional neural network model. In: International conference on Asian digital libraries. Cham: Springer; 2017. p. 30–40.
40. Hassan SU, Imran M, Iftikhar T, Safder I, Shabbir M. Deep stylometry and lexical & syntactic features-based author attribution on Plops digital repository. In: International conference on Asian digital libraries. 2017; Springer, pp. 119–127.
41. Shardlow M, Batista-Navarro R, Thompson P, Nawaz R, McNaught J, Ramanaidu S. Identification of research hypotheses and new knowledge from scientific literature. BMC Med Informat Decis Making. 2018;18(1):46.
42. Siegelman, Lourie, Power, Ammar. Extracting scientific figures with distantly supervised neural networks. In: Proceedings of the18thACM/IEEE on joint conference on digital libraries. 2018; pp. 223–232. ACM.
43. Siegel N, Horvitz Z, Levin R, Farhadi A. Figure seer: parsing result-figures in research papers. In: European Conference on Computer Vision. Berlin: Springer; 2016. p. 664–80.
44. Sinoara RA, Camacho-Collados J, Rossi RG, Navigli R, Rezende SO. Knowledge-enhanced document embeddings for text classification. Knowl Based Syst. 2019;163:955–71.
45. Sunder Meyer M, Schlüter R, Ney H. LSTM neural networks for language modeling. In: Thirteenth annual conference of the international speech communication association. 2012.
46. Suzuki T, Fuji A. Mathematical document categorization with structure of mathematical expressions. In: 2017 ACM/IEEE Joint Conference on Digital Libraries (JCDL). 2017; pp. 1–10. IEEE.
47. Taarab S, Bhatia S, Mitra P, Giles CL. Algorithm Seer: a system for extracting and searching for algorithms in scholarly big data. IEEE Trans Big Data. 2016;2(1):3–17.
48. Taarab S, Mitra P, Giles CL. A hybrid approach to discover semantic hierarchical sections in scholarly documents. In: 2015 13th international conference on document analysis and recognition (ICDAR). 2015; pp. 1081–1085. IEEE.
49. Wang X, Rak R, Restiform A, Nowata C, Rupp CJ, Batista-Navarro TB, Raheel N, Ramanaidu S. Detecting experimental techniques and selecting relevant documents for protein-protein interactions from biomedical literature. BMC Bioinformatics. 2011;12(S11).

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.